DOCKER NETWORKING WITH ISOLATED CONTAINERS

DEVOPS REPORT
(22ITF02-Project Work)

Submitted by

GOWRI D
(23ITR048)

in partial fulfilment of the requirements for the

award of the degree

of

BACHELOR OF TECHNOLOGY IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

APRIL 2026

DEPARTMENT OF INFORMATION TECHNOLOGY
KONGU ENGINEERING COLLEGE

(Autonomous)
PERUNDURAI, ERODE – 638060
APRIL 2026

BONAFIDE CERTIFICATE

This is to certify that this devops report entitled **"Docker Networking with Isolated Containers"** is the bonafide work carried out by **GOWRI D (23ITR048)** in partial fulfilment of the requirements for the course **22ITF02 – DEVOPS** during the academic year **2025 – 2026**, and that the work has been carried out under my instructor and has not been submitted elsewhere for any other course or award.

COURSE INSTRUCTOR **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the Final Project Review for the course 22ITF02 – DEVOPS held on

_____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF INFORMATION TECHNOLOGY KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

APRIL 2026

DECLARATION

I affirm that the Project Report titled **"Docker Networking with Isolated Containers"** being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by me. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

GOWRI D
(23ITR048)

Date:

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date: Name and Signature of the Instructor with seal

ABSTRACT

This project focuses on designing and implementing a secure multi-container application using Docker networking concepts. The system consists of frontend, backend, and database services, where network isolation is achieved using custom Docker bridge networks. The backend acts as a secure bridge between the public-facing frontend and the private database, ensuring controlled communication. The project demonstrates how container isolation improves security, prevents unauthorized access, and follows real-world cloud architecture practices similar to Virtual Private Cloud (VPC). The implementation highlights container orchestration using Docker Compose, internal DNS-based communication, and automated testing to verify network isolation, thereby bridging academic learning with industry-level DevOps practices.

ACKNOWLEDGEMENT

First and foremost, I acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

I express my gratefulness to Correspondent **THIRU. E. R. K. Krishnan, M.Com.** and all the trust members of Kongu Vellalar Institute of Technology Trust for providing all the necessary facilities to complete the project successfully.

I express my deep sense of gratitude to our beloved Principal **Dr.R.Parameshwaran M.E.,Ph.D.,** for providing us an opportunity to complete the project.

I express my gratitude to **Dr. S. Anandamurugan M.E.,Ph.D.,** Head of the Department, Department of Information Technology for his valuable suggestions.

I am highly indebted to **Fg Offr. A. Jeevanantham M.S., Assistant Professor (Selection Grade)** Department of Information Technology for his valuable supervision and advice for the fruitful completion of the project.

I am thankful to the faculty members of the Department of Information Technology for their valuable guidance and support.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**iv**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**8**
	1.1 SCOPE	
	1.2 OBJECTIVE	
2	**SYSTEM SPECIFICATIONS**	**9**
	2.1 HARDWARE REQUIREMENTS	
	2.2 SOFTWARE REQUIREMENTS	
	2.3 TOOLS SPECIFICATION	
3	**SYSTEM DESIGN**	**11**
	3.1 FLOW CHART	
4	**IMPLEMENTATION**	**13**
5	**RESULT AND CONCLUSION**	**15**
	5.1 RESULT	
	5.2 CONCLUSION	
	5.3 TOOL CREDENTIALS	
	APPENDIX 1 : CODING	**21**
	APPENDIX 2: COURSE CERTIFICATE	**24**
	REFERENCES	

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
3.1	FLOW CHART	11
5.1	DOCKER DESKTOP	15
5.2	DOCKER NETWORKS (APP & DB-NETWORKS)	15
5.3	COMMAND PROMPT	16
5.4	LANDING PAGE	17
5.5	DOCKER HUB DASHBOARD	17
5.6	GITHUB DASHBOARD	19
5.7	AWS DASHBOARD	19
5.8	DOCKER PROFILE	20

CHAPTER 1

INTRODUCTION

1.1 SCOPE

The scope of this project is to design and implement a secure multi-container application using Docker networking with isolated containers. The system focuses on dividing the application into multiple layers such as frontend, backend, and database, and isolating them using custom Docker networks to ensure controlled communication. The backend acts as a secure bridge between the public-facing frontend and the private database, preventing direct access and enhancing security. The project demonstrates how network segmentation improves application safety, reliability, and maintainability. It also highlights the use of Docker Compose for managing multi-container environments and simulates real-world cloud architectures similar to Virtual Private Cloud (VPC). Overall, the scope emphasizes secure container communication and modern DevOps practices using Docker.

1.2 OBJECTIVES

The main objective of this project is to implement Docker networking with isolated containers to ensure secure and controlled communication between application services. The project aims to divide the application into frontend, backend, and database layers and connect them using custom Docker networks. Another objective is to restrict direct access to the database by isolating it within a private network, allowing communication only through the backend. The project also focuses on using Docker Compose to manage multi-container applications efficiently. Additionally, it aims to verify network isolation through testing and demonstrate how such architecture improves security, reliability, and scalability. Overall, the objective is to provide practical knowledge of secure container-based application deployment using modern DevOps practices.

CHAPTER 2

SYSTEM SPECIFICATIONS

2.1 HARDWARE REQUIREMENTS

- A computer or laptop with at least Intel i5 / AMD equivalent processor to efficiently run Docker containers.
- Minimum 8 GB RAM (recommended 16 GB) to support multiple containers running simultaneously.
- At least 50 GB free disk space for storing Docker images, containers, and project files.
- Stable internet connection for pulling Docker images and accessing GitHub repositories.
- 64-bit operating system (Windows 10/11, Linux, or macOS) with virtualization support (WSL2 or equivalent) for smooth Docker execution.

2.2 SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11 (64-bit) with WSL2 enabled, or Linux/macOS with Docker support.
- **Docker Desktop:** Used to build, run, and manage containerized applications and networks.
- **Docker Compose:** Used to define and manage multi-container applications and custom networks.
- **GitHub:** Used for version control and storing project source code.
- **Node.js & npm:** Used for developing and running the backend and frontend applications.
- **MongoDB:** Used as the database for storing application data.
- **Nginx:** Used as a reverse proxy server to handle frontend requests and route API calls.
- **Text Editor/IDE:** Visual Studio Code or equivalent for writing code and configuration files.

2.3 TOOL SPECIFICATION

2.3.1 Github

GitHub is a widely used platform for version control and source code management. It allows developers to store, track, and manage code changes using Git.

In this project, GitHub was used to maintain the complete source code of the Track Placement Preparation application, including frontend, backend, and Docker configuration files. It helped in version tracking, collaboration, and maintaining project history. GitHub also ensured that the codebase

remains organized and easily accessible throughout the development lifecycle.

2.3.2 Docker

Docker is an open-source platform used to build, run, and manage applications inside containers.

In this project, Docker was used to containerize the frontend, backend, and database services. Each component runs in an isolated container, ensuring consistency across environments. Docker also enables efficient resource usage and simplifies deployment. By using Docker, the application becomes portable and can run on any system without dependency issues.

2.3.3 Docker Compose

Docker Compose is a tool used to define and manage multi-container Docker applications. In this project, Docker Compose was used to orchestrate multiple services such as frontend, backend, and database. It defines custom networks (app-network and db-network), volumes, and service dependencies in a single file. This makes it easy to start and manage the entire application using a single command. It also plays a key role in implementing network isolation between containers.

2.3.4 MongoDB

MongoDB is a NoSQL database used to store data in a flexible, document-based format.In this project, MongoDB was used as the database to store student data and application information. It runs inside a Docker container and is placed in a separate network (db-network) to ensure security. The database is not directly accessible from the frontend, which enhances data protection.

2.3.5 Node.js and Express.js

Node.js and Express.js are used to build the backend server.In this project, Node.js and Express.js handle API requests, business logic, and communication with the database. The backend acts as a secure bridge between the frontend and the database. It is connected to both networks (app-network and db-network), allowing controlled communication while maintaining isolation.

CHAPTER 3

SYSTEM DESIGN

3.1 FLOW CHART

The flow chart represents the overall workflow of the project, starting from the development of the application and committing the source code to GitHub. The application is containerized using Docker, where separate images are created for frontend, backend, and database services. These containers are then managed using Docker Compose, which defines the services, volumes, and custom networks. Two isolated networks are created. The frontend is served using Nginx and communicates with the backend, while the backend interacts with the database.

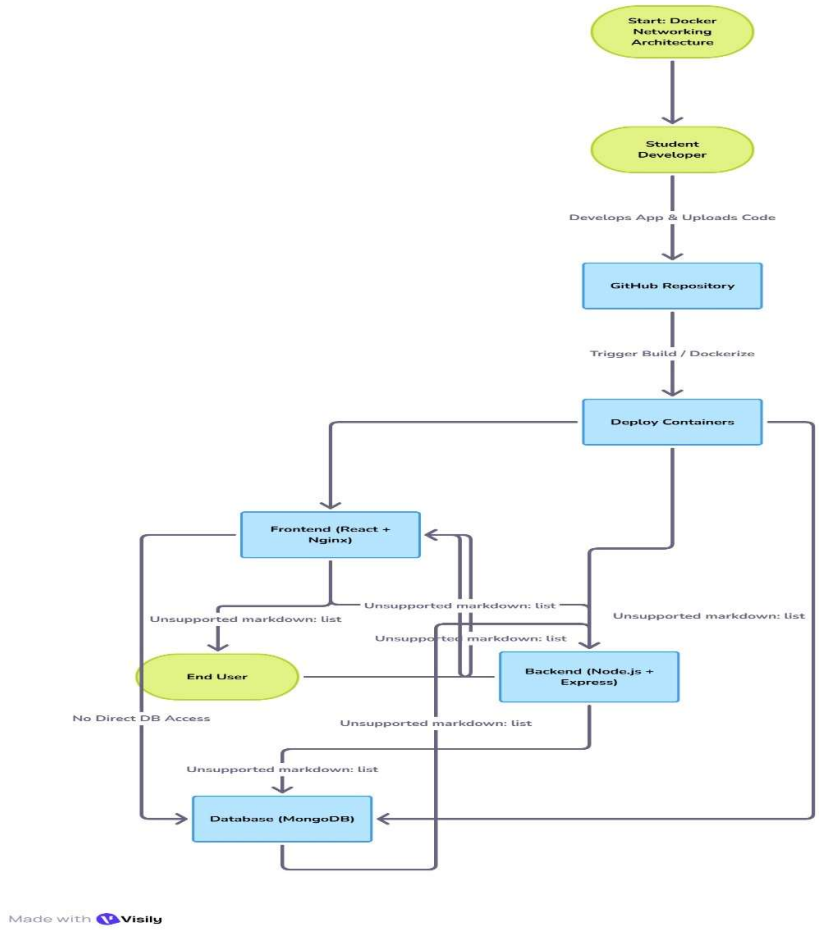


Fig 3.1: Flow chart

The flow chart illustrates the complete workflow of the project across different design layers. In the Application Layer, the application is developed, tested, and version-controlled using GitHub. In the Containerization Layer, the application components are packaged into Docker images, ensuring portability and consistency across environments. In the Networking Layer, custom networks are created to isolate the database from the frontend, while allowing controlled access through the backend. Moving to the Execution Layer, Docker Compose is used to run and manage all containers efficiently. Finally, in the Access Layer, the application is exposed to users through the frontend interface, and requests are processed securely through the backend to the database. This architecture ensures proper isolation, improved security, reliable communication, and efficient handling of application workflows, aligning with modern DevOps practices

CHAPTER 4

IMPLEMENTATION

1. Application Development

• A full-stack web application (Track Placement Preparation) was developed to demonstrate secure container-based architecture.

• The frontend was built using React.js and the backend was developed using Node.js with Express.js.

• The application was tested locally to ensure proper functionality.

• Source code was maintained and version-controlled using GitHub.

2. Backend and Database Setup

• The backend server was configured to handle API requests and business logic.

• MongoDB was used as the database to store application data.

• The backend was connected to the database using secure connection methods.

• Proper routing and data handling mechanisms were implemented for smooth communication.

3. Containerization with Docker

• Docker was used to containerize all components of the application.

• Separate Dockerfiles were created for frontend and backend services.

• Each service was built into a Docker image and tested independently.

• Containerization ensured consistency, portability, and isolation of application components.

4. Multi-Container Management using Docker Compose

• Docker Compose was used to manage multiple containers in a single configuration file.

• Services such as frontend, backend, and database were defined in the compose file.

• Dependencies between services were managed using depends_on.

• The entire application was started using a single command, simplifying deployment.

5. Network Isolation and Security

• Two custom Docker networks were created:

- **app-network** (Frontend ↔ Backend)
- **db-network** (Backend ↔ Database)

• The frontend container was restricted from directly accessing the database.

• The backend acted as a secure bridge between frontend and database.

- This network segmentation ensured strong isolation and improved security.

6. Reverse Proxy and Request Handling

- Nginx was used to serve the frontend application.
- It also acted as a reverse proxy to route API requests to the backend.
- This setup improved performance and simplified communication between services.

7. Testing and Verification

- The application was accessed through a web browser to verify functionality.
- Docker commands such as docker network inspect were used to verify network isolation.
- A test script was executed to ensure frontend cannot access the database directly.
- The system was validated for security, reliability, and proper communication flow.

ADVANTAGES

- **Security:** Network isolation prevents unauthorized access to the database.
- **Consistency:** Docker ensures the application runs uniformly across different environments.
- **Portability:** The containerized application can run on any system without configuration issues.
- **Scalability Ready:** The architecture can be easily extended for scaling in future.
- **Efficient Resource Usage:** Containers are lightweight and optimized.
- **Controlled Communication:** Backend acts as a secure bridge between services.
- **Easy Deployment:** Docker Compose allows managing the entire system with a single command.
- **Reliability:** Isolation reduces chances of system failure due to component issues.
- **Practical Learning:** Provides hands-on experience with real-world DevOps architecture and networking concepts.

CHAPTER 5

RESULT AND CONCLUSION

5.1 RESULT

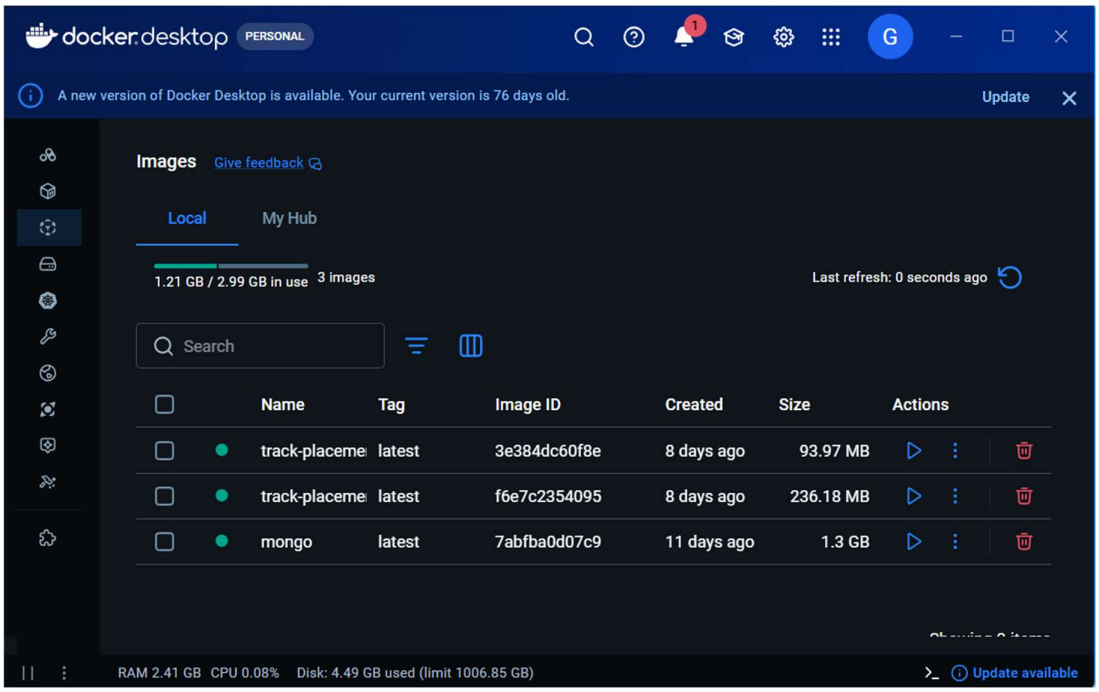


Fig 5.1: Docker Desktop

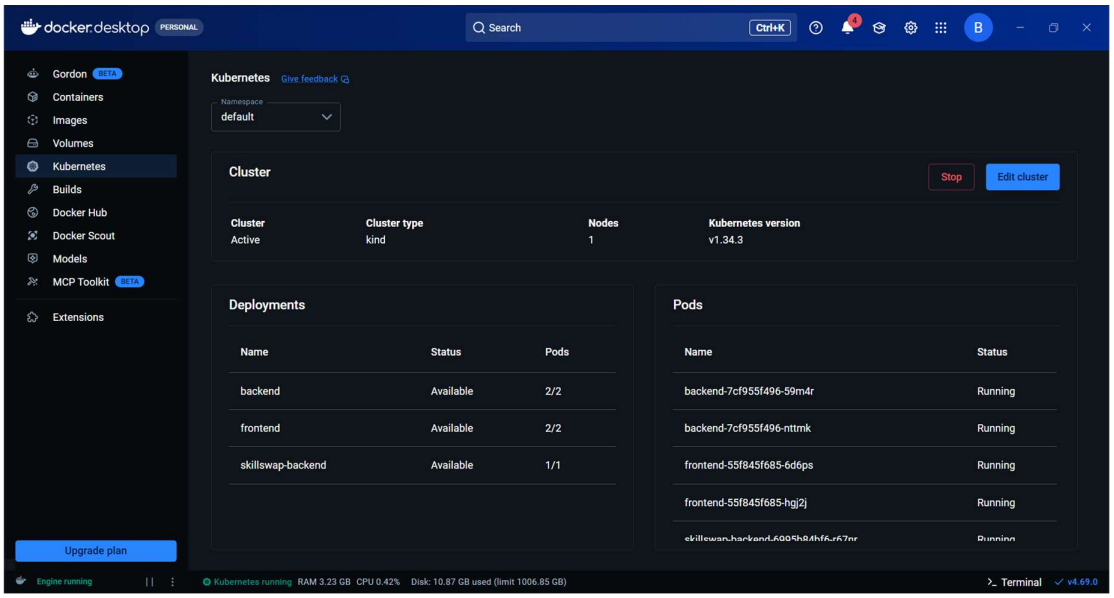


Fig 5.2: Docker Networks (app-network & db-network)

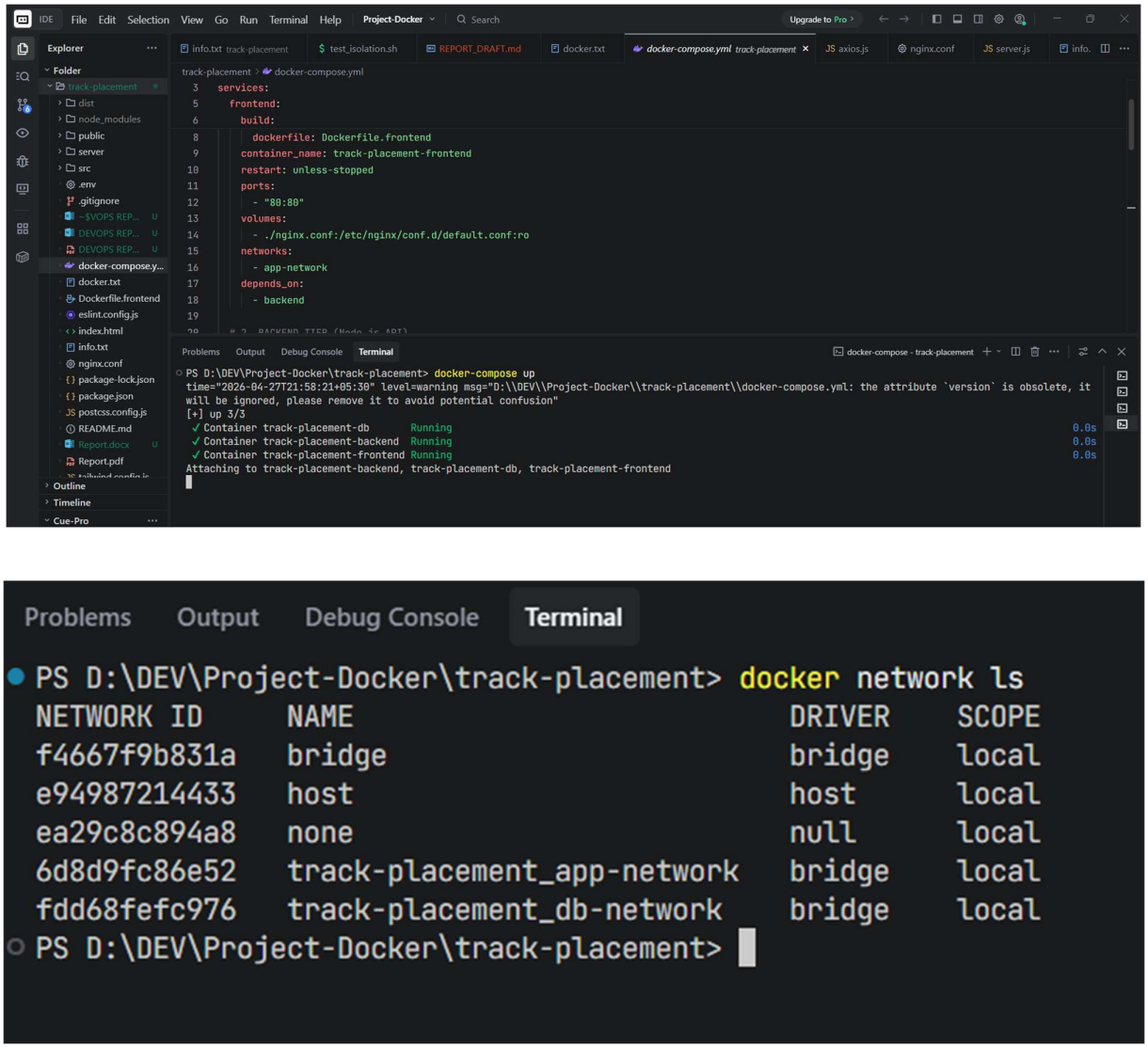


Fig 5.3: Command Prompt Terminal

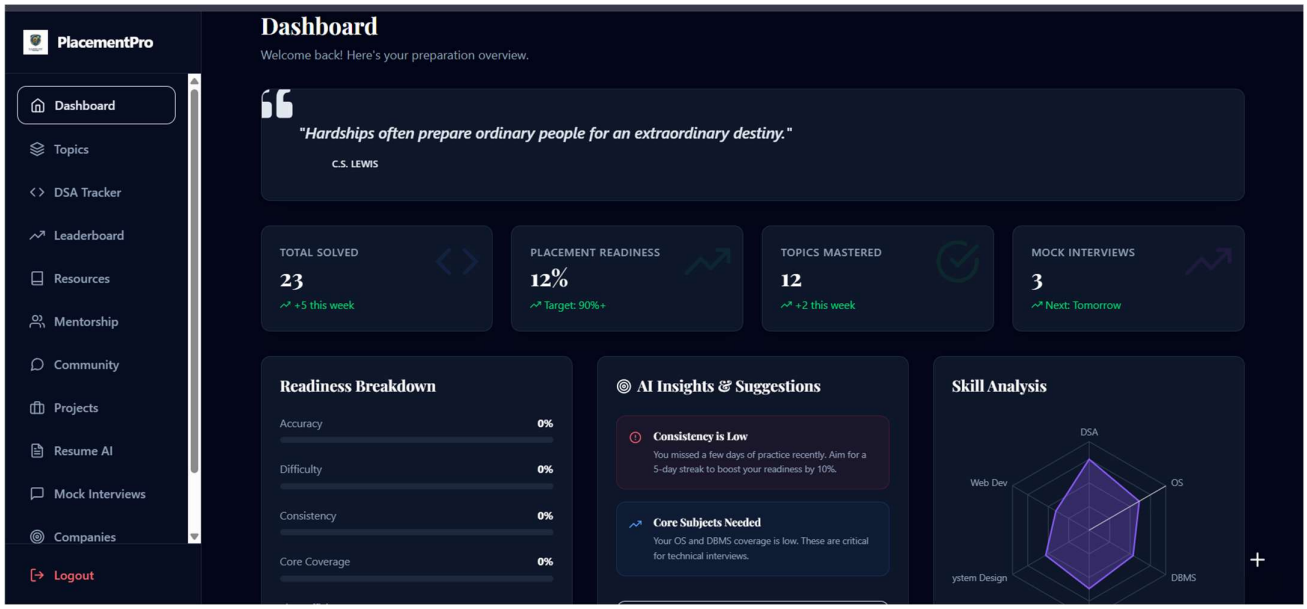


Fig 5.4 : Landing Page

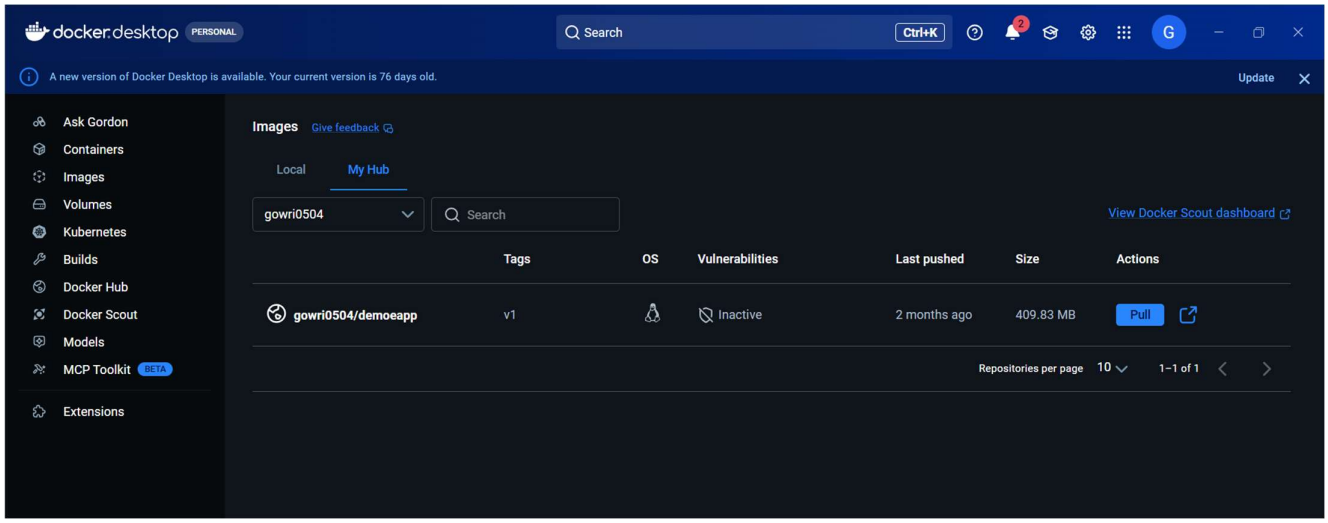


Fig 5.5 : Docker Hub Dashboard

5.2 CONCLUSION

This project successfully demonstrated the design and implementation of a secure multi-container application using Docker networking with isolated containers. By using Docker, the application components such as frontend, backend, and database were containerized, ensuring consistency and portability across different environments. Docker Compose was used to efficiently manage multiple containers and define service dependencies, simplifying the deployment process.

The project focused on implementing network isolation using custom Docker networks, namely app-network and db-network. The frontend was allowed to communicate only with the backend, while the database was restricted to backend access, preventing direct exposure and improving security. This architecture ensured controlled communication between services and reduced the risk of unauthorized access.

The implementation also highlighted the importance of container-based architecture in modern application development. By separating services into individual containers, the system achieved better modularity, reliability, and ease of maintenance. Testing and verification confirmed that network isolation was properly enforced, and all components functioned as expected.

Overall, this project provided valuable hands-on experience in Docker, container networking, and multi-container management using Docker Compose. It demonstrated how secure and efficient application deployment can be achieved using modern DevOps practices. The knowledge gained from this project is essential for building scalable, secure, and production-ready applications in real-world environments.

.

5.3 TOOL CREDENTIALS

5.3.1 Github

ID – Gowri0504

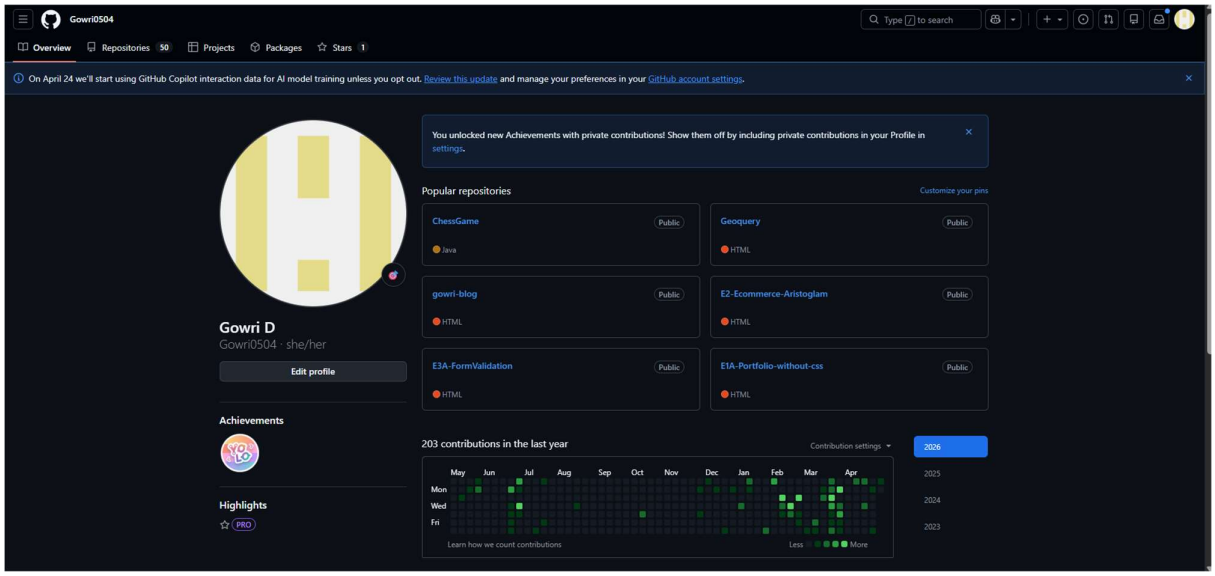


Fig 5.6 – Github Dashboard

5.3.2 AWS

ID - 205500198343

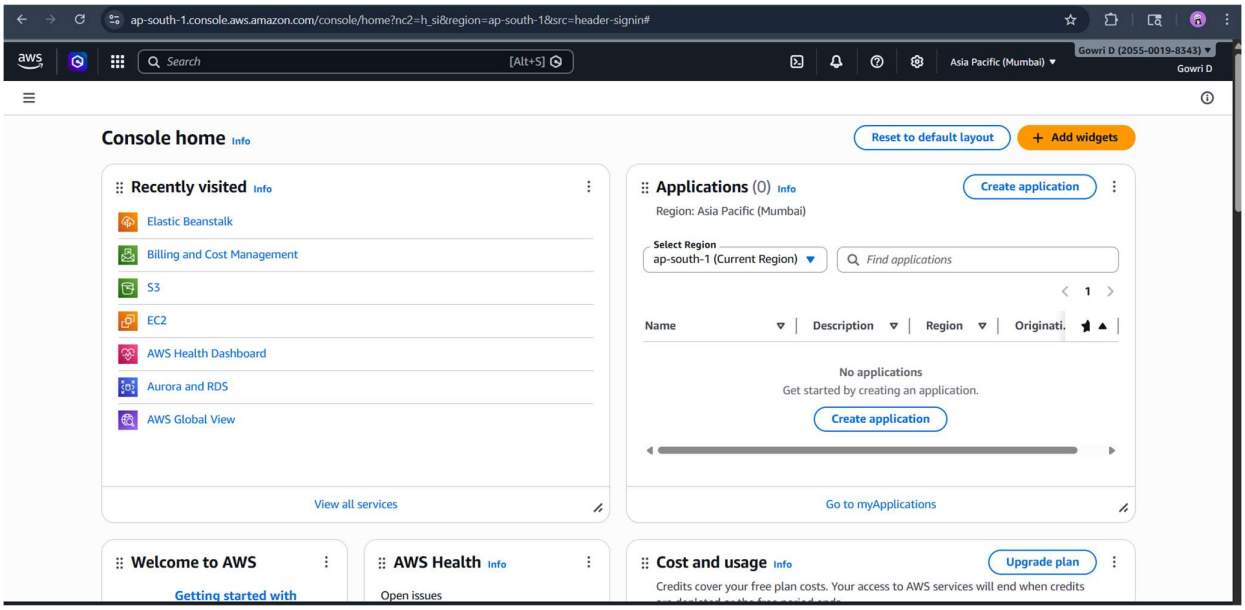


Fig 5.7 – AWS Dashboard

5.3.3 Docker Hub

ID - gowri0504

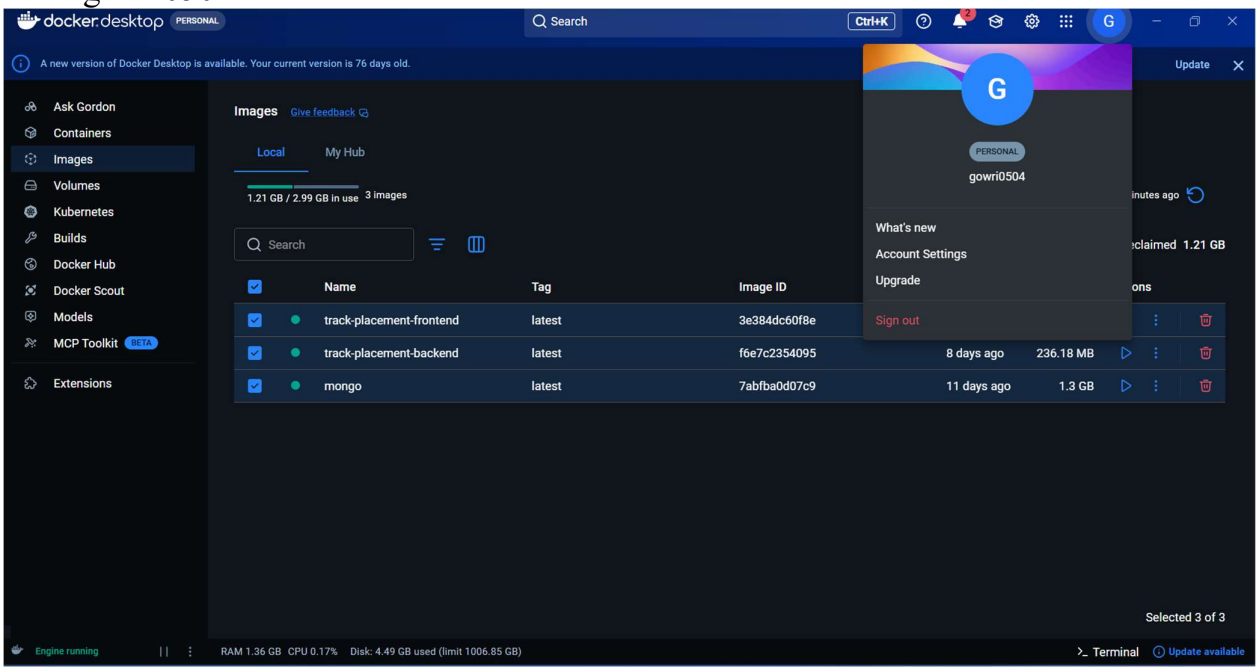


Fig 5.8 – Docker Profile

APPENDIX 1

CODING

DockerFile
(Frontend)

A Dockerfile is used to define how an application is built and executed inside a Docker container. In this configuration, a Node.js base image is used to set up the environment. The application files are copied into the working directory, dependencies are installed, and the application is built using npm commands. The built application is then served using a lightweight server on port 3000. This ensures consistency and portability across different environments.

```
# Build stage
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker-compose.yml

Docker Compose is used to define and manage multi-container applications. In this project, it is used to run frontend, backend, and database services with proper network isolation using custom networks.

```
version: '3.8'

services:
  # 1. FRONTEND TIER (Nginx + React App)
  frontend:
    build:
      context: .
```

```
    dockerfile: Dockerfile.frontend
  container_name: track-placement-frontend
  restart: unless-stopped
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  networks:
    - app-network
  depends_on:
    - backend

# 2. BACKEND TIER (Node.js API)
backend:
  build:
    context: ./server
    dockerfile: Dockerfile
  container_name: track-placement-backend
  restart: unless-stopped
  environment:
    - NODE_ENV=production
    - MONGO_URI=mongodb://db:27017/track-placement
    - PORT=3000
  networks:
    - app-network
    - db-network
  depends_on:
    - db

# 3. DATABASE TIER (MongoDB)
db:
  image: mongo:latest
  container_name: track-placement-db
  restart: unless-stopped
  volumes:
```

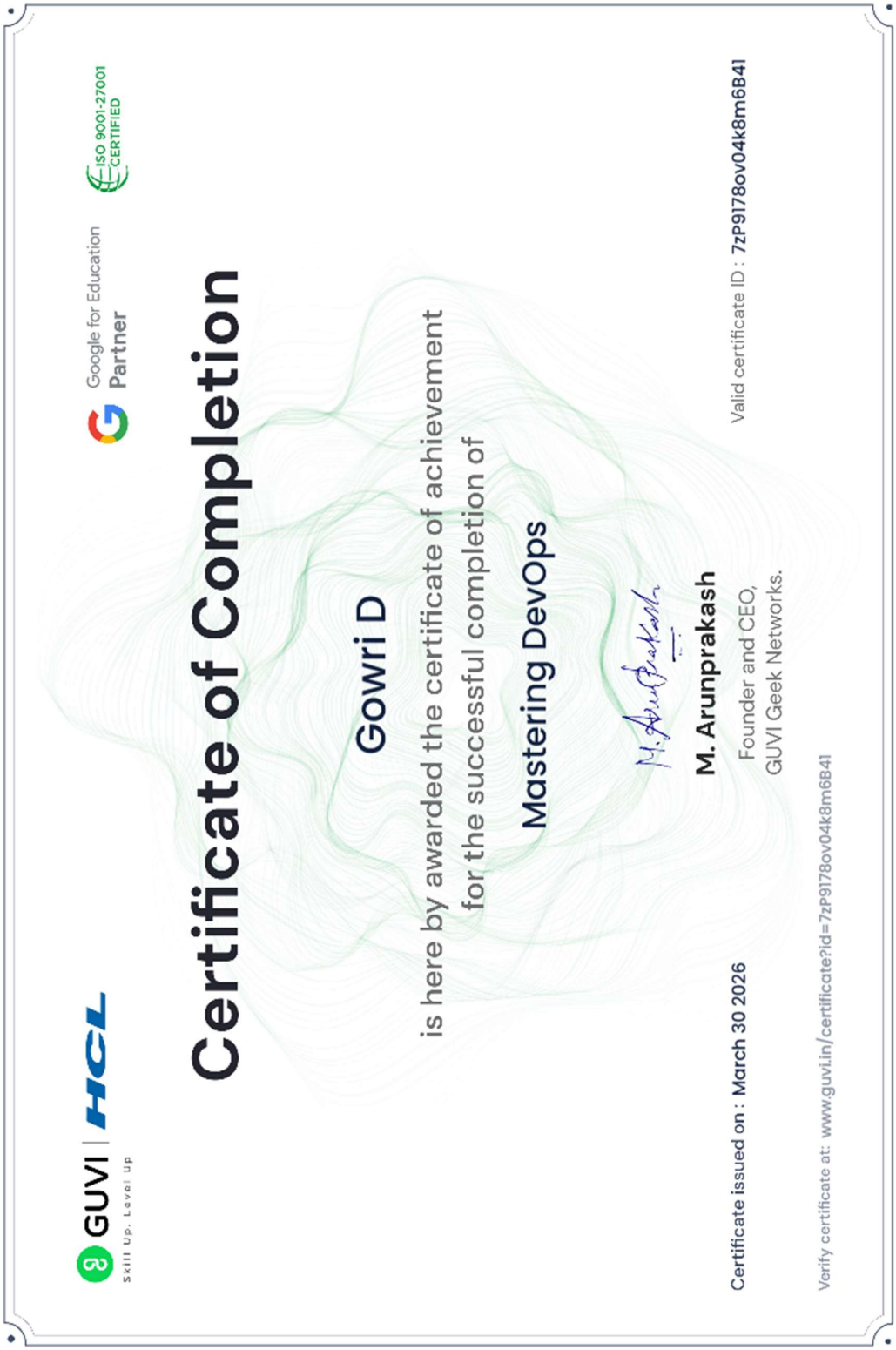
```
      - db-data:/data/db
    networks:
      - db-network
    # DATABASE IS ISOLATED - NO PORTS EXPOSED TO HOST

networks:
  # Multi-tier Network Isolation
  app-network:
    driver: bridge
  db-network:
    driver: bridge

volumes:
  # Secure Data Persistence
  db-data:
    driver: local
```

APPENDIX 2

MASTERING DEVOPS COURSE COMPLETION CERTIFICATE



GITHUB LINK: https://github.com/Gowri0504/Devops-Project.git

REFERENCES:

- Docker Documentation – https://docs.docker.com/
- Docker Compose Documentation – https://docs.docker.com/compose/
- Docker Networking Documentation – https://docs.docker.com/network/
- Docker CLI Reference – https://docs.docker.com/engine/reference/commandline/docker/
- GitHub Documentation – https://docs.github.com/
- Node.js Documentation – https://nodejs.org/en/docs/
- Express.js Documentation – https://expressjs.com/
- MongoDB Documentation – https://www.mongodb.com/docs/
- NGINX Documentation – https://nginx.org/en/docs/
- Visual Studio Code Documentation – https://code.visualstudio.com/docs
- Docker Hub Documentation – https://docs.docker.com/docker-hub/
- Docker Bridge Network Guide – https://docs.docker.com/network/bridge/
- Docker Compose File Reference – https://docs.docker.com/compose/compose-file/
- REST API Design Guide – https://restfulapi.net/
- MDN Web Docs – https://developer.mozilla.org/